



DISTRIBUTED SMART TRAFFIC LIGHT CONTROL SYSTEM

**Malika Khalimarden
Vladislav Solodov**

INTRODUCTION

Project goal: To create a decentralized traffic management system where each intersection autonomously adapts to traffic flow and cooperates with neighboring ones to eliminate congestion and delays in real time.



CHOICE OF TOOLS

1. VSCode



2. Node.js



3. MQTT



4. JSON



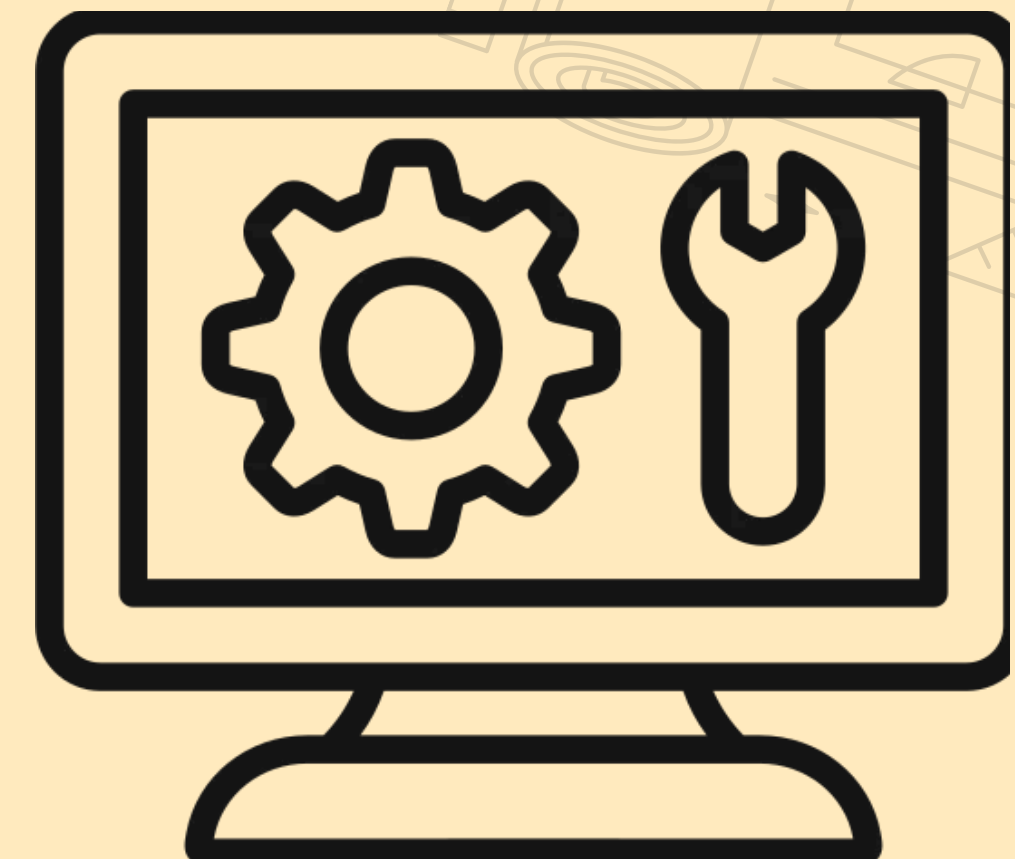
5. Grafana



6. Influxdb



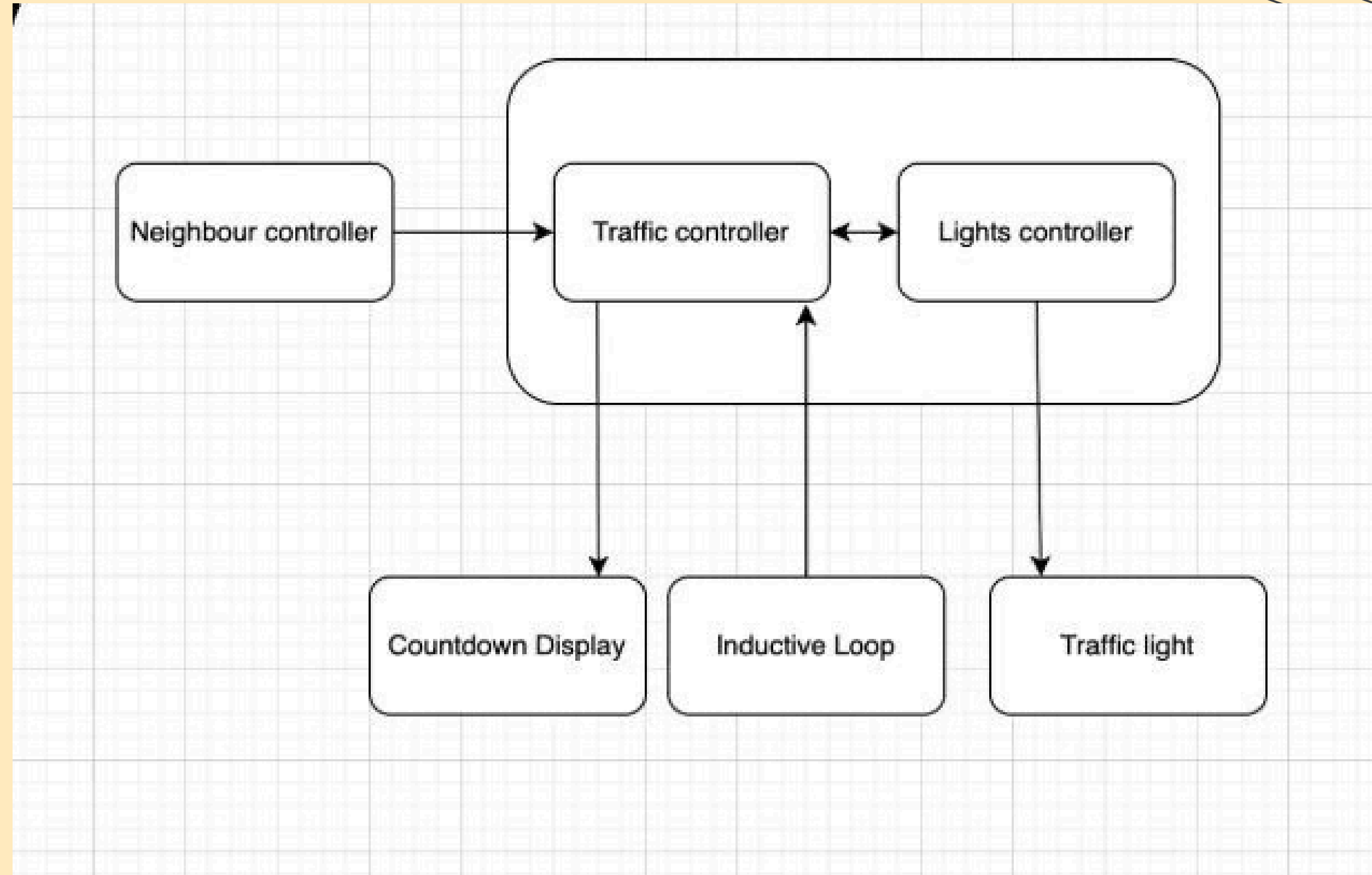
7. Docker



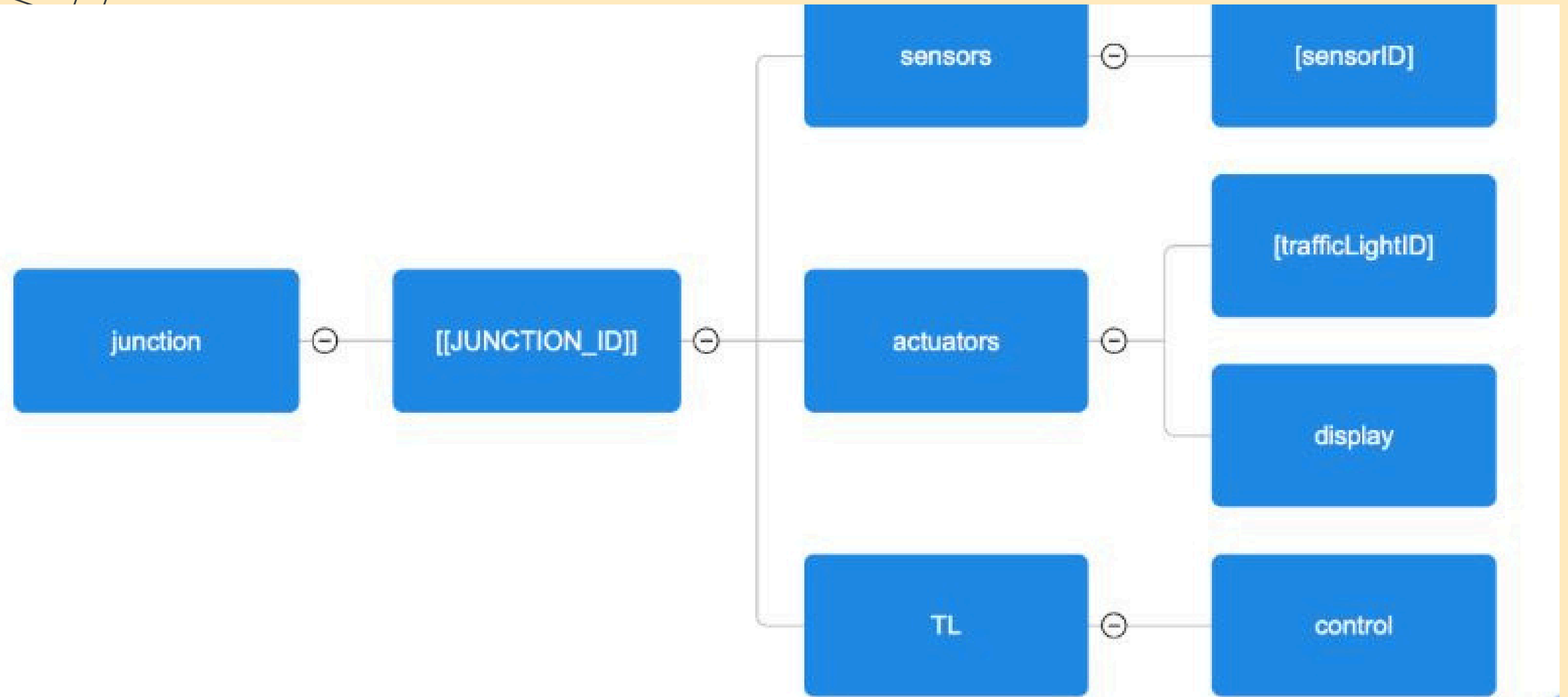
IMPLEMENTED CONTROLLERS

COMPONENTS COMMUNICATION

(all via mqtt)

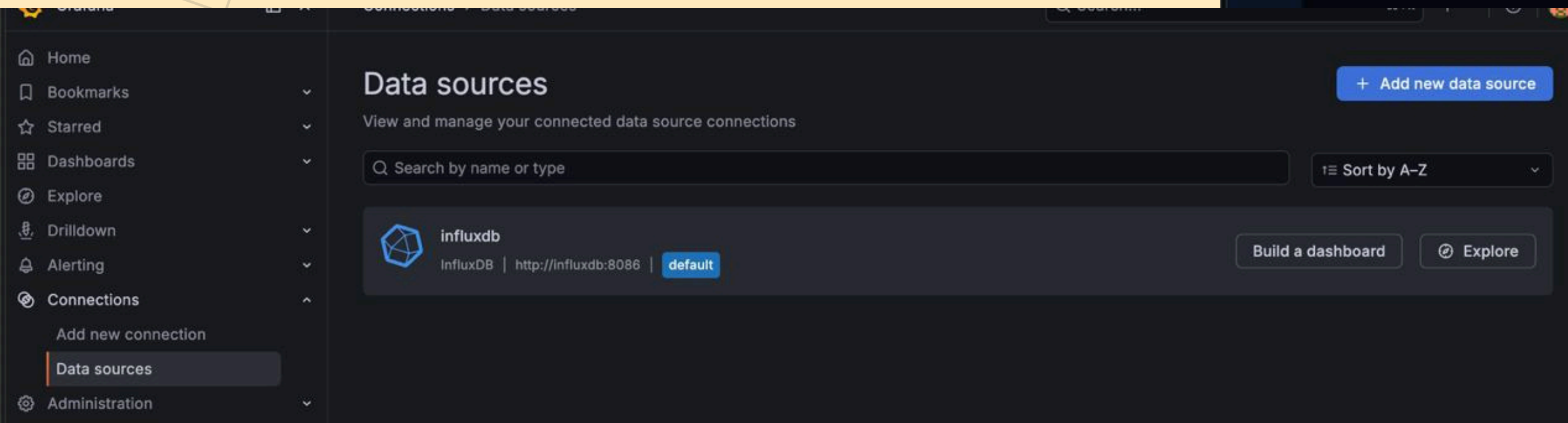
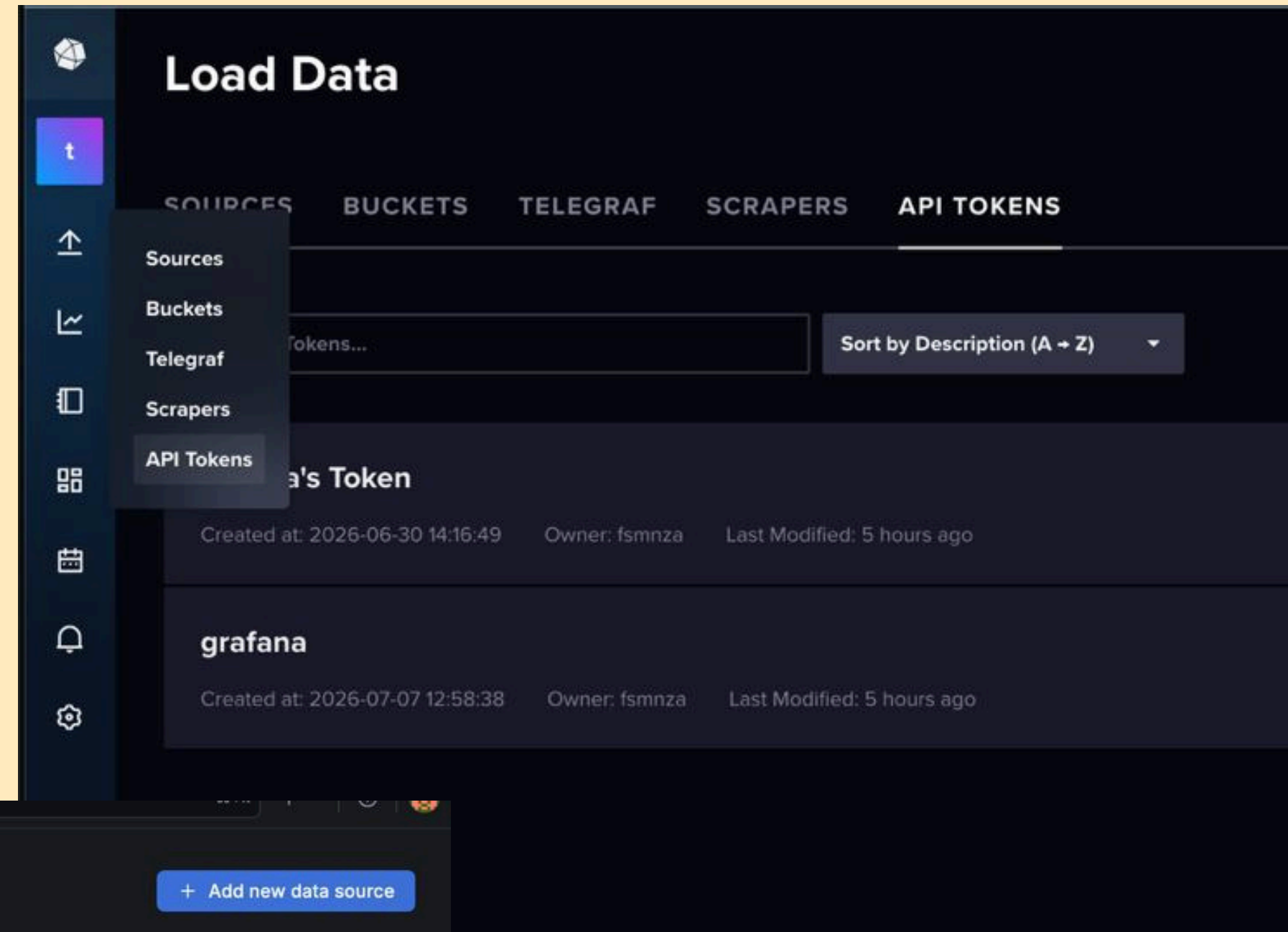
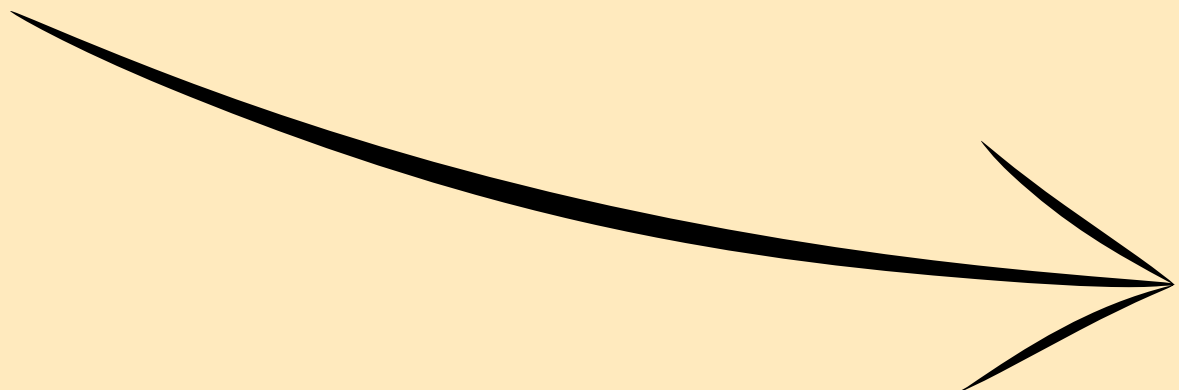


MQTT TOPICS

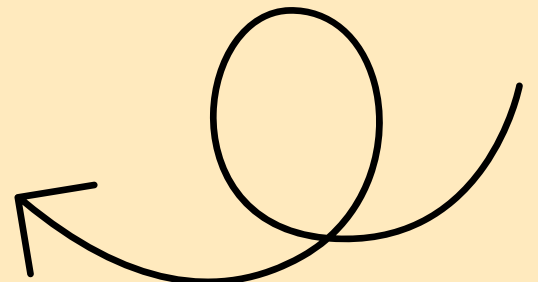


INFLUXDB & GRAFANA

Get InfluxDb Token

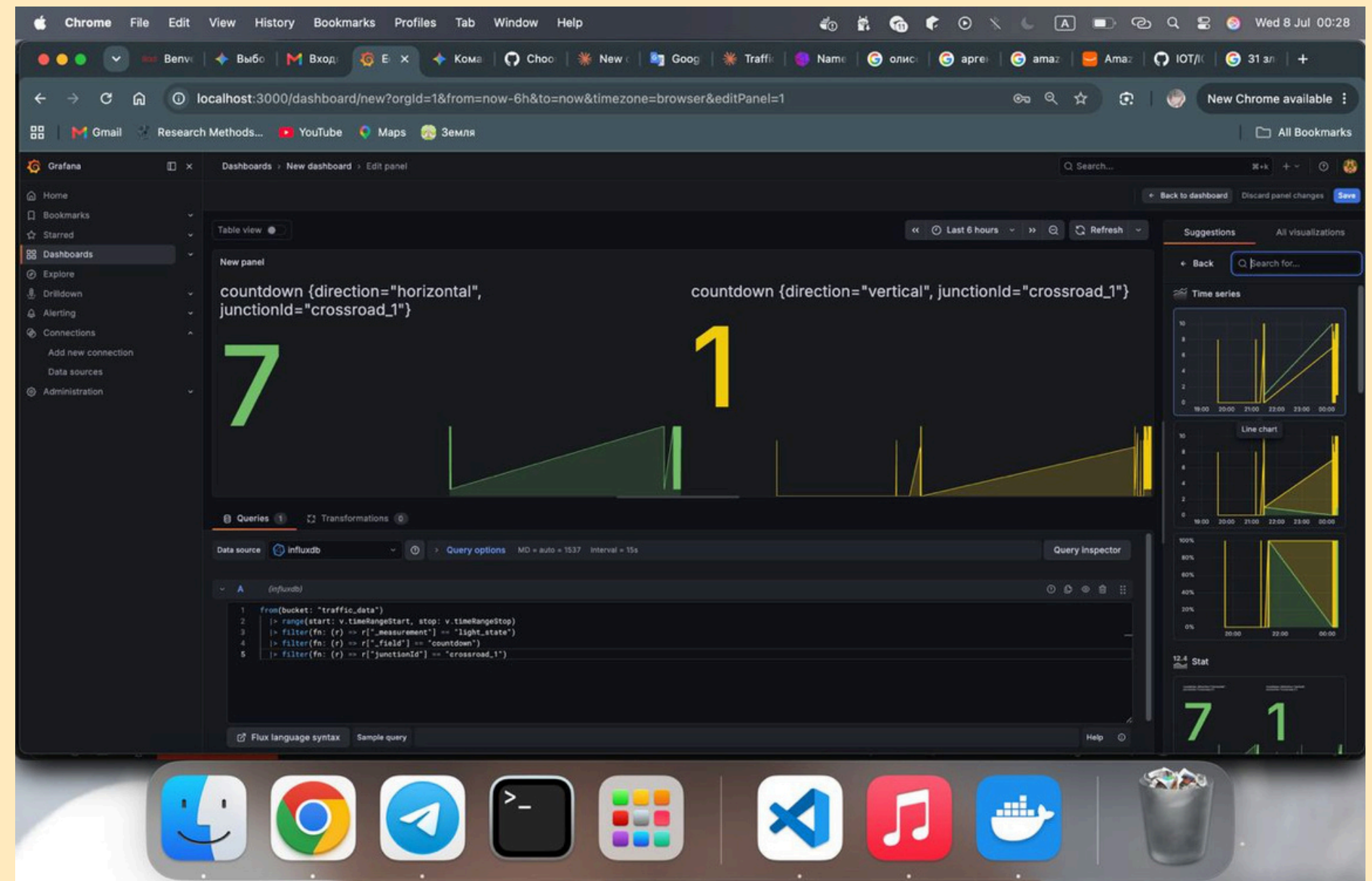


Add data source



INFLUXDB

GRAFANA



DOCKER

Upped the docker

```
malikakhalimarden@Malikas-MacBook-Air IOT % docker compose up -d
[+] Running 4/4
 ✓ Network iot_default          Created
 ✓ Container mqtt_broker        Started
 ✓ Container influxdb           Started
 ✓ Container grafana_dashboard  Started
```

iot

/Users/malikakhalimarden/Desktop/IOT

mqtt_broker

eclipse-mosquitto:2

Running

1883:1883

Show all ports (2)

grafana_dashboard

grafana/grafana-oss

Running

3000:3000

influxdb

influxdb:2.7

Running

8086:8086

2026-07-08 02:12:19 grafana_dashboard | logger=plugin.installer t=2026-07-07T20:12:19.108036796Z level=info msg="Installing plugin" pluginId=grafana-exploretraces-app version=

2026-07-08 02:12:19 grafana_dashboard | logger=plugin.installer t=2026-07-07T20:12:19.108036796Z level=info msg="Installing plugin" pluginId=grafana-exploretraces-app version=

2026-07-08 02:12:19 grafana_dashboard | logger=installer.fs t=2026-07-07T20:12:19.171851546Z level=info msg="Downloaded and extracted grafana-exploretraces-app v2.1.0 zip successfully to /var/lib/grafana/plugins/grafana-exploretraces-app"

2026-07-08 02:12:19 grafana_dashboard | logger=plugins.registration t=2026-07-07T20:12:19.188547713Z level=info msg="Plugin registered" pluginId=grafana-exploretraces-app

2026-07-08 02:12:19 grafana_dashboard | logger=plugin.backgroundinstaller t=2026-07-07T20:12:19.188568504Z level=info msg="Plugin successfully installed" pluginId=grafana-exploretraces-app version= duration=1.124966834s

2026-07-08 02:12:19 grafana_dashboard | logger=plugin.backgroundinstaller t=2026-07-07T20:12:19.188710338Z level=info msg="Installing plugin" pluginId=grafana-metricsdrilldown-app version=

2026-07-08 02:12:21 grafana_dashboard | logger=plugin.installer t=2026-07-07T20:12:21.232116339Z level=info msg="Installing plugin" pluginId=grafana-metricsdrilldown-app version=

2026-07-08 02:12:21 grafana_dashboard | logger=installer.fs t=2026-07-07T20:12:21.318474297Z level=info msg="Downloaded and extracted grafana-metricsdrilldown-app v2.2.0 zip successfully to /var/lib/grafana/plugins/grafana-metricsdrilldown-app"

2026-07-08 02:12:21 grafana_dashboard | logger=plugins.registration t=2026-07-07T20:12:21.333411089Z level=info msg="Plugin registered" pluginId=grafana-metricsdrilldown-app

2026-07-08 02:12:21 grafana_dashboard | logger=plugin.backgroundinstaller t=2026-07-07T20:12:21.333433297Z level=info msg="Plugin successfully installed" pluginId=grafana-metricsdrilldown-app version= duration=2.144707918s

SIMULATION

```
1
2  const { spawn } = require('child_process');
3  const path = require('path');
4
5  const junctionIdArg = process.argv[2] || 'crossroad_1';
6
7  const scripts = [
8    { name: 'TLC', path: 'traffic_controller/traffic_controller.js' },
9    { name: 'Crossroad', path: 'crossroad_1.js' },
10   { name: 'DBLogger', path: 'db_logger/db_logger.js' },
11   { name: 'IL', path: 'inductive_loop/cars_over_il.js' },
12 ];
13
14 const runningProcesses = [];
15
16 function startScript(script) {
```

We have simulation0.js. And we just run it. And it runs another js class

TRAFFIC

Traffic is the
smart traffic
light switches
phases using
sensors.

```
traffic_controller > JS traffic_controller.js > ...
> console Aa ab .* ? of 4 ↑ ↓ ≡ ×

1  const mqtt = require('mqtt');
2  const client = mqtt.connect('mqtt://localhost:1883');
3
4  const JUNCTION_ID = process.argv[2] || 'crossroad_1';
5
6  client.on('connect', () => {
7    console.log(`[TLC] Smart controller started for ${JUNCTION_ID}. Listening to sensors...`);
8    client.subscribe(`junction/${JUNCTION_ID}/il/#`);
9  });
10
11 client.on('message', (topic, message) => {
12   try {
13     const carData = JSON.parse(message.toString());
14     console.log(`[TLC] Smart controller started for ${JUNCTION_ID}. Listening to sensors...`);
15
16     if (carData.direction === 'horizontal') {
17       console.log(`[TLC] On the horizontal road, there is a traffic jam! Sending command to switch lights...`);
18
19       const command = {
20         command: 'Switch light',
21         requestPhase: 'horizontal'
22       };
23
24       client.publish(`junction/${JUNCTION_ID}/tl/control`, JSON.stringify(command));
25     }
26   } catch (e) {
27     console.error(`[TLC] Error processing sensor data:', e);
28   }
29 });
```


CROSSROAD



```
JS crossroad_1.js X JS db_logger.js {} info.json New Prolead Project JS traffic_controller.js JS receiver.js
JS crossroad_1.js > [🔍] activeDirection
1  const mqtt = require('mqtt');
2  const client = mqtt.connect('mqtt://localhost:1883');
3
4  const junctionIdArg = process.argv[2] || 'crossroad_1';
5  const JUNCTION_ID = junctionIdArg;
6
7  const DEFAULT_GREEN_MS = 10000;
8  const YELLOW_MS = 3000;
9  const ALL_RED_MS = 1000;
10
11 const VEHICLE_DIFF_THRESHOLD = 5;
12 let vehicleCounts = { horizontal: 0, vertical: 0 };
13
14 let activeDirection = 'vertical';
15 let phase = 'GREEN';
16 let phaseStart = Date.now();
17 let currentGreenDurationMs = DEFAULT_GREEN_MS;
18 let pendingSwitchTo = null;
19 let lastLoggedCountdown = null;
20
21 const CONTROL_TOPIC = `junction/${JUNCTION_ID}/tl/control`;
22 const TRAFFIC_COUNT_TOPIC = `junction/${JUNCTION_ID}/traffic/count`;
23
24 client.on('connect', () => {
25   console.log(`[Actuator] Connected. Managing physical lights for ${JUNCTION_ID}`);
26   client.subscribe(CONTROL_TOPIC, (err) => {
27     if (err) console.error('[Actuator] Subscribe error:', err);
28   });
29   client.subscribe(TRAFFIC_COUNT_TOPIC, (err) => {
30     if (err) console.error('[Actuator] Subscribe error (traffic count):', err);
31   });
32   setInterval(tick, 500);
33   publishStatus();
34 });
35
36 client.on('error', (err) => {
37   console.error('[Actuator] MQTT Client Error:', err);
```

The traffic
light machine
switches
phases
according to
traffic jams.

CARS_OVER_IL

Simulation of vehicle motion sensors at an intersection.

```
inductive_loop > JS cars_over_il.js > ...
1  const InductiveLoop = require('./InductiveLoop');
2
3  const junctionIdArg = process.argv[2] || 'crossroad_1';
4
5
6  let sensors = [];
7  let directions = ['vertical', 'horizontal']
8  let places = ['infrontof', 'behind'];
9  let c = 0;
10 for (let i of directions)
11 {
12   for (let j of places)
13   {
14     for (let q = 0; q<2; q+=1)
15     {
16       let direction = i.concat(c);
17       let sensor = new InductiveLoop({
18         brokerUrl: 'mqtt://localhost:1883',
19         junctionId: junctionIdArg,
20         sensorId: direction,
21         place: j
22       });
23       c+=1;
24       sensors.push(sensor)
25       //console.log(sensor);
26     }
27   }
28 }
29 c = 0;
30 }
31
```


**THANK
you**

